

# Driving the Buzzer

## Experiment Objective

In this experiment, you will learn how to drive the passive buzzer on the MicroROS-V2 control board. Through this example, you will master the configuration of the ESP32's LEDC (PWM) peripheral, as well as how to use PWM signals to drive the passive buzzer to produce sound and adjust its frequency.

## Experiment Principle

### 1. Keywords

| Keyword                      | Description                                                                                                             |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Passive buzzer               | Contains no internal oscillator; requires an external PWM square wave to drive it to produce sound                      |
| PWM (Pulse Width Modulation) | Controls the pitch and volume of the buzzer by adjusting the frequency and duty cycle of the square wave                |
| LEDC                         | The ESP32's PWM controller peripheral; timers, channels, and resolution can be configured                               |
| Frequency                    | The frequency of the PWM square wave, which determines the pitch of the buzzer (range 100~10000 Hz)                     |
| Duty cycle                   | The ratio of the PWM high-level time to the whole period, which determines the volume of the buzzer                     |
| Timer resolution             | The number of bits that determine the PWM precision, e.g. 10-bit resolution corresponds to a duty cycle range of 0~1023 |

### 2. Technical Architecture

The program uses the ESP32's LEDC PWM controller to drive the passive buzzer. During initialization, the LEDC timer and channel are configured and GPIO47 is set as the PWM output. In the main loop, `Beep_On_Time(500)` is called every 1 second to make the buzzer sound for 500 ms and then stop automatically. A background task calls `Beep_Handle()` every 10 ms to manage the buzzer state and implement delayed automatic shutdown. Key files are located in `~/esp/base_samples/beep/`, including `main/main.c` (main program entry), `components/beep/beep.h` (pin definitions and function declarations) and `components/beep/beep.c` (buzzer driver implementation).

# Experiment Steps

## 1. Hardware Preparation

| Hardware                  | Requirement |
|---------------------------|-------------|
| MicroROS-V2 control board | ✓ Required  |
| USB Type-C data cable     | ✓ Required  |

## 2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Confirm the buzzer on the control board is functioning properly

## 3. Compile and Flash

```
cd ~/esp/base_samples/beep
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

## 4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

# Experiment Result

After compiling and flashing successfully, the serial monitor will output:

```
hello yahboom
I (xxx) BEEP: Start Beep_Task with core:1
I (xxx) MAIN: Nice to meet you!
```

At the same time, the buzzer on the control board **beeps once every 1 second**, each time lasting **500 milliseconds**, producing a clear "beep" sound.

## Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/beep/main/main.c`
- Buzzer driver: `~/esp/base_samples/beep/components/beep/beep.h`, `beep.c`

# 1. Main Program Logic

main/main.c is the program entry, and the flow is as follows:

- 1. Print the startup message "hello yahboom" and a welcome log
- 2. Call Beep\_Init() to initialize the buzzer PWM
- 3. Wait 1 second
- 4. Enter the main loop, calling Beep\_On\_Time(500) every 1 second to make the buzzer sound for 500 ms and then stop automatically

# 2. Key Components

components/beep/beep.h — pin definitions and function declarations:

Parameter definition file: ~/esp/base\_samples/beep/components/beep/beep.h

```
// Buzzer GPIO pin (schematic: Buzzer -> U6-24 -> GPIO47)
#define BEEP_GPIO 47

// Passive buzzer default parameters
#define BEEP_DEFAULT_FREQ_HZ 2000 // Default frequency 2 KHz
#define BEEP_DEFAULT_DUTY_PCT 50 // Default duty cycle 50%

#define BEEP_STATE_OFF 0 // Off state
#define BEEP_STATE_ON_ALWAYS 1 // Always-on state
#define BEEP_STATE_ON_DELAY 2 // Delayed off state
```

components/beep/beep.c — buzzer driver implementation, with the following core functions:

| Function                        | Description                                                                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Beep_Init()                     | Initialize the LEDC PWM timer and channel, and create the buzzer background task                                                               |
| Beep_On()                       | Turn on the buzzer for continuous sound                                                                                                        |
| Beep_Off()                      | Turn off the buzzer                                                                                                                            |
| Beep_On_Time(uint16_t time)     | Set the buzzer sound duration (in milliseconds); it turns off automatically after the timeout. time=0 turns it off, time=1 sounds continuously |
| Beep_Set_Freq(uint16_t freq_hz) | Set the buzzer frequency (range 100~10000 Hz)                                                                                                  |
| Beep_Get_Freq()                 | Get the current frequency                                                                                                                      |
| Beep_Handle()                   | Buzzer state handler, called every 10 ms                                                                                                       |

PWM initialization parameters:

The buzzer uses the ESP32's LEDC PWM controller, configured as follows:

Source file: `~/esp/base_samples/beep/components/beep/beep.c`

```
// LEDC timer configuration
ledc_timer_config_t timer_conf = {
    .speed_mode      = LEDC_LOW_SPEED_MODE,
    .duty_resolution = LEDC_TIMER_10_BIT,      // 10-bit resolution (0~1023)
    .timer_num       = LEDC_TIMER_0,
    .freq_hz         = BEEP_DEFAULT_FREQ_HZ,   // Default 2 KHz
    .clk_cfg         = LEDC_AUTO_CLK
};

// LEDC channel configuration
ledc_channel_config_t channel_conf = {
    .gpio_num        = BEEP_GPIO,              // GPIO47
    .speed_mode      = LEDC_LOW_SPEED_MODE,
    .channel         = LEDC_CHANNEL_0,
    .intr_type       = LEDC_INTR_DISABLE,
    .timer_sel       = LEDC_TIMER_0,
    .duty            = 0,
    .hpoint          = 0
};
```

**Frequency and duty cycle control:**

```
// Start PWM output
Beep_PWM_Start(freq_hz, duty_percent); // Frequency range limited to 100~10000 Hz
// Stop PWM output (set duty cycle to 0)
Beep_PWM_Stop();
```

**Delayed shutdown working mechanism:**

After `Beep_On_Time(500)` is called, the buzzer state switches to `BEEP_STATE_ON_DELAY`. The background task `Beep_Task` calls `Beep_Handle()` every 10 ms, decrementing the `beep_on_time` counter. When the count reaches zero, `Beep_Off()` is called automatically to turn off the buzzer.

## Common Problems

| Problem                     | Cause                                                           | Solution                                                                                                    |
|-----------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Buzzer does not sound       | The buzzer pin is not GPIO47                                    | Check the <code>BEEP_GPIO</code> macro in <code>beep.h</code>                                               |
| Abnormal or distorted sound | Incorrect PWM frequency setting                                 | The default frequency is 2000 Hz; adjust it with <code>Beep_Set_Freq()</code>                               |
| Buzzer keeps sounding       | <code>Beep_Off()</code> not called after <code>Beep_On()</code> | Use <code>Beep_On_Time(ms)</code> instead, or make sure <code>Beep_Off()</code> is called at the right time |

| Problem                              | Cause                                     | Solution                                                                                                |
|--------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Compilation error:<br>ledc undefined | Missing driver<br>component<br>dependency | Check whether <code>REQUIRES driver</code> in<br><code>components/beep/CMakeLists.txt</code> is correct |